

Paging

- Con paging si ha solo frammentazione interna
- Calcolo della frammentazione interna
 - Esempio:
 - ▶ Page size = 2,048 bytes
 - ▶ Process size = 72,766 bytes
 - ▶ 35 pages + 1,086 bytes
 - ▶ Frammentazione interna di $2,048 - 1,086 = 962$ bytes
 - Frammentazione worst case = 1 frame – 1 byte
 - In media frammentazione = $1 / 2$ frame size
 - Quindi piccoli frame desiderabili?
 - ▶ ... ma ogni entry della page table impegna memoria per tracciarla
 - Le dimensioni delle pagine crescono nel tempo
 - ▶ Solaris supporta due dimensioni – 8 KB e 4 MB
 - ▶ Win 11 supporta 4 KB, 2 MB, 1 GB
 - ▶ Linux supporta default (4 KB) e huge pages

Paging

- Con paging si ha solo frammentazione interna
 - Le dimensioni delle pagine crescono nel tempo
 - ▶ Linux supporta default (4 KB) e huge pages (def 2 MB)

scrive il valore della conf variable

```
$getconf PAGESIZE  
4096
```

Statistiche di mem in kb

```
$grep Huge /proc/meminfo
```

```
AnonHugePages:          0 kB  
ShmemHugePages:         0 kB  
FileHugePages:          0 kB  
HugePages_Total:        0  
HugePages_Free:         0  
HugePages_Rsvd:         0  
HugePages_Surp:         0  
Hugepagesize:           2048 kB  
Hugetlb:                0 kB
```

Esercizio

- Dato un sistema con uno spazio di indirizzi logici a 14 bit e pagine da 2KB quante entry avrà la tabella delle pagine?

Esercizio

- Dato un sistema con uno spazio di indirizzi logici a 14 bit e pagine da 2KB quante entry avrà la tabella delle pagine?
- Con pagine di 2KB avremo 2^{11} B quindi $n = 11$
- Cioè 11 bit per offset, dunque rimangono 3 bit per indicare la pagina
- La tabella delle pagine avrà 2^3 entry

Esercizio

- Si assuma di avere un sistema con uno spazio degli indirizzi logici di 15 bit e 8 pagine. Quanto sono grandi le pagine del sistema?

Esercizio

- Si assuma di avere un sistema con uno spazio degli indirizzi logici di 15 bit e 8 pagine. Quanto sono grandi le pagine del sistema?
- Pagine $8 = 2^3$, cioè 3 bit per rappresentarle
- Bit di offset $15 - 3 = 12$
- 2^{12} byte da indirizzare, cioè pagine da 4 KB ($2^2 \times 2^{10}$)

Esercizio

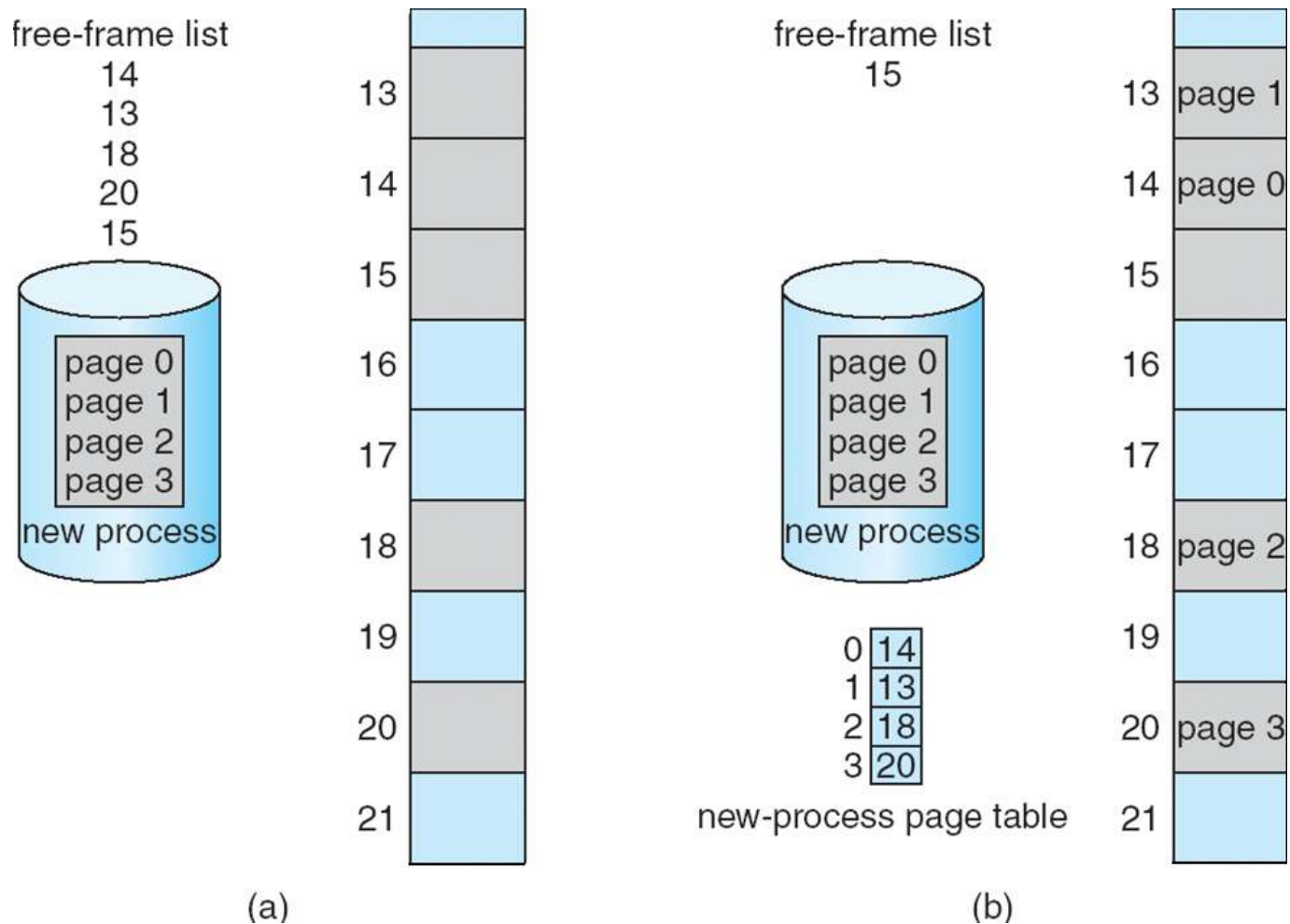
- Sia dato un sottosistema di memoria con paginazione, caratterizzato dalle seguenti dimensioni:
 - Frame di 4 MB e memoria fisica indirizzabile di 128 GB
- Si calcoli il numero di bit minimo per indicizzare tutte le pagine associate

Esercizio

- Sia dato un sottosistema di memoria con paginazione, caratterizzato dalle seguenti dimensioni:
 - Frame di 4 MB e memoria fisica indirizzabile di 128 GB
 - Si calcoli il numero di bit minimo per indicizzare tutte le pagine associate
-
- Frame 4 MB, quindi $2^2 \times 2^{20} = 2^{22}$ byte
 - Memoria fisica 128 GB = $2^7 \times 2^{30} = 2^{37}$ byte
 - Numero frame = $2^{37} / 2^{22} = 2^{15}$
-
- Quindi 15 bit è il numero minimo per indicizzare le pagine

Frame Liberi

Un processo da eseguire viene valutato in pagine e se richiede n pagine occorrono n frame liberi in memoria. Le pagine sono via via allocate e associate ai frame liberi



Before allocation

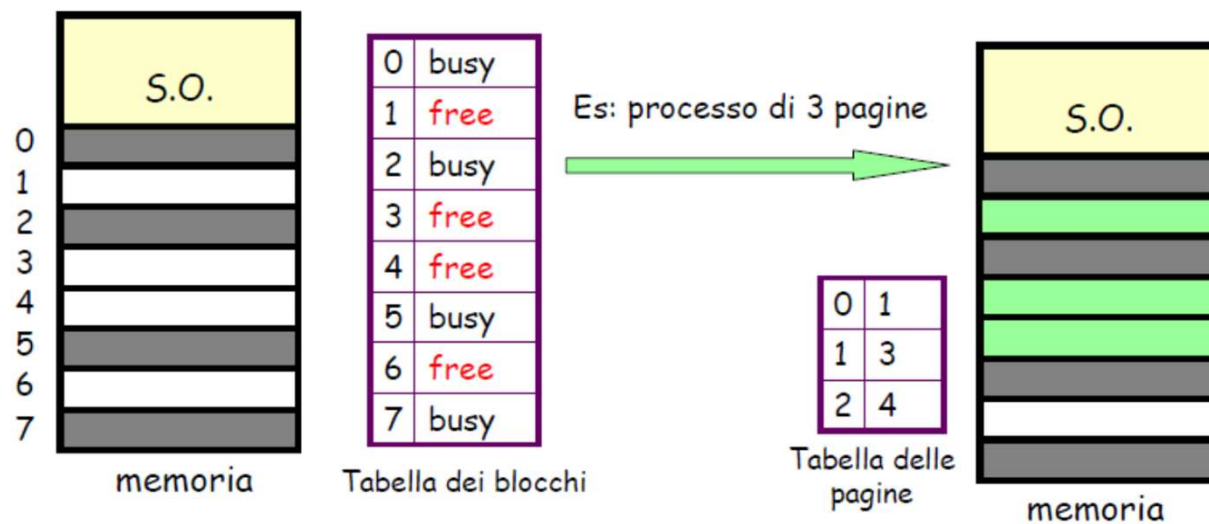
After allocation

Frame Liberi

Un processo da eseguire viene valutato in pagine e se richiede n pagine occorrono n frame liberi in memoria. Le pagine sono via via allocate e associate ai frame liberi

Il Sistema Operativo mantiene una Tabella dei Blocchi per tracciare l'uso della memoria (libera, occupata, assegnazione)

Quando un processo chiede memoria viene consultata la tabella dei blocchi per l'assegnazione

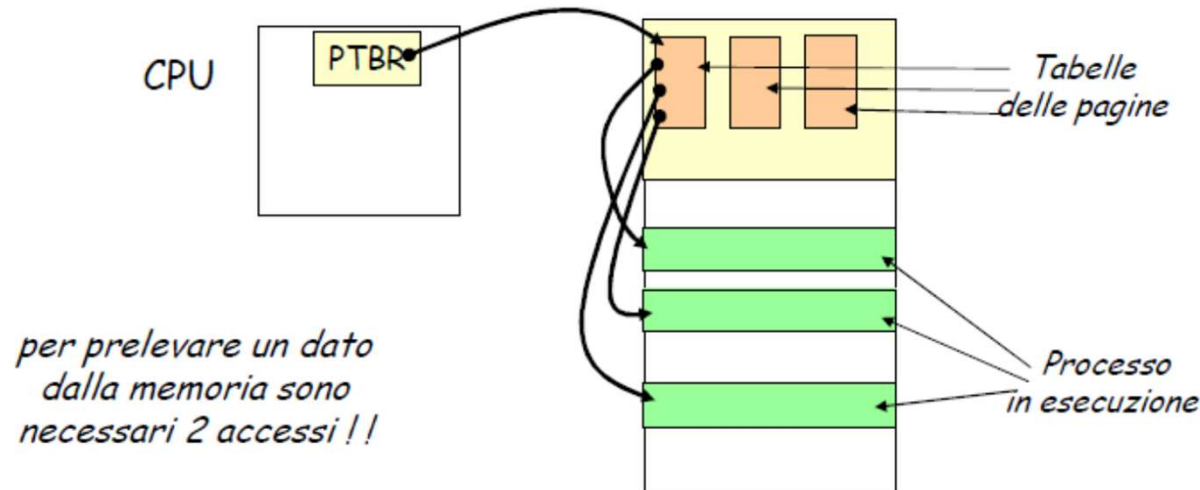


Before allocation

After allocation

Implementazione della Page Table

- ❑ Struttura per-processo con puntatore nel PCB
- ❑ Quando un processo va in esecuzione devono essere ricaricati i registri e ricaricati i valori per la gestione della page table
- ❑ La page table può essere implementata in vari modi
 - ❑ Registri dedicati caricati dal dispatcher della CPU, accesso del SO in modalità privilegiata, richiede gestione context switch, piccola tabella (es. 256 entry)
 - ❑ Se più grande (es. 2^{20} entry) la page table è mantenuta in memoria principale
 - ▶ **Page-table base register (PTBR)** punta alla page table
 - ▶ **Page-table length register (PTLR)** indica la dim della page table



Implementazione della Page Table

- ❑ Struttura per-processo con puntatore nel PCB
- ❑ Quando un processo va in esecuzione devono essere ricaricati i registri e ricaricati i valori per la gestione della page table
- ❑ La page table può essere implementata in vari modi
 - ❑ Registri dedicati caricati dal dispatcher della CPU, accesso del SO in modalità privilegiata, richiede gestione context switch, piccola tabella (es. 256 entry)
 - ❑ Se più grande (es. 2^{20} entry) la page table è mantenuta in memoria principale
 - ▶ **Page-table base register (PTBR)** punta alla page table
 - ▶ **Page-table length register (PTLR)** indica la dim della page table
 - ▶ Veloce context switch (cambia il puntatore)
 - ▶ Lento accesso a memoria. Ogni accesso a dati/istruzioni richiede due accessi in memoria: uno per la page table ed uno per i dati/istruzioni
 - ▶ I due accessi in memoria possono essere risolti utilizzando una cache speciale detta **registri associativi** o **translation look-aside buffers (TLBs)**

Memoria Associativa (TLB)

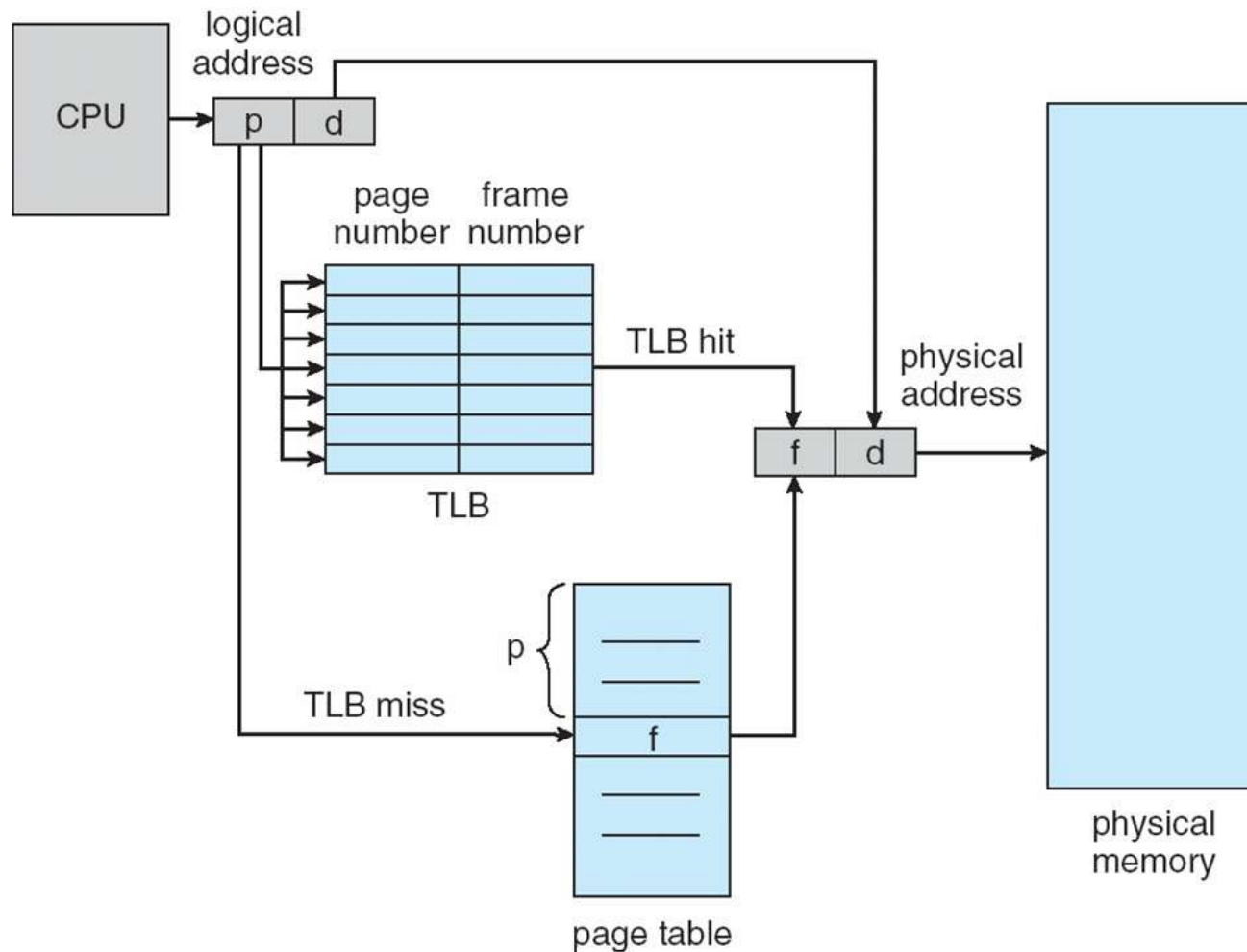
- ❑ Per la traduzione di un indirizzo si cerca prima in memoria associativa e poi in memoria principale
- ❑ Memoria associativa – ricerca parallela

Page #	Frame #

- ❑ Traduzione di un indirizzo (p, d)
 - ❑ Se p è in un registro associativo, genera il frame # in output
 - ❑ Altrimenti (TLB miss) prendi il frame # dalla page table in memoria
 - ▶ La nuova coppia trovata può essere aggiunta nei registri

Hardware per Paging con TLB

Hardware necessario per implementare il paging con **Translation Look-aside Buffers (TLBs)**



Implementazione della Page Table

- Ogni volta che non si trova il valore in TLB (TLB miss), il valore è caricato sulla TLB per un accesso veloce la volta successiva
 - Se piena occorrono politiche di rimpiazzo
 - ▶ Least recently used (LRU), round robin, random
 - Alcune entry possono essere **cablate** per consentire l'accesso permanente
 - ▶ Es. Codice del kernel rilevante

- Alcune TLB hanno **address-space identifiers (ASIDs)** per ogni entry
 - Identifica univocamente il processo
 - Permette di mantenere info su diversi processi contemporaneamente
 - ▶ Altrimenti dovrebbe fare il flush per ogni context switch per evitare di far accedere il processo alle associazioni della page table di un processo precedente
 - Fornisce una protezione del suo spazio di indirizzi
 - ▶ Quando risolve l'indirizzo verifica se il processo corrente corrisponde a quello indicato, se non corrisponde si considera una TLB miss

Tempo di Accesso Effettivo

- Assumiamo ε unità di tempo la ricerca in TLB
 - Può essere $< 10\%$ del tempo di accesso in memoria
- Assumiamo la hit ratio = α
 - Hit ratio – percentuale di volte che si trova la pagina in TLB (dipende dal numero di registri)
 - ▶ Es. 80% significa 80% delle volte si trova la pagina
- Consideriamo $\alpha = 80\%$, $\varepsilon = 20\text{ns}$ per una ricerca in TLB e $T_a = 100\text{ns}$ per un accesso in memoria
- **Effective Access Time (EAT)**
$$\begin{aligned} \text{EAT} &= (T_a + \varepsilon) \alpha + (2 T_a + \varepsilon)(1 - \alpha) \\ &= 2 T_a + \varepsilon - T_a \alpha \end{aligned}$$
 - Se $\alpha = 80\%$, $\varepsilon = 20\text{ns}$ per la ricerca in TLB search, $T_a = 100\text{ns}$ per accesso in memoria ... $\text{EAT} = 0.80 \times 100 + 0.20 \times 200 + 20 = 140\text{ns}$
 - Con un più realistico hit ratio $\rightarrow \alpha = 99\%$, $\varepsilon = 20\text{ns}$ per la ricerca in TLB, 100ns per accesso in memoria ... $\text{EAT} = 0.99 \times 100 + 0.01 \times 200 + 20 = 121\text{ns}$

Tempo di Accesso Effettivo

- Nei sistemi moderni più complicato il calcolo perché più livelli di TLB
 - Es. La CPU Intel Core i7 ha una TLB L1 da 128-entry per le istruzioni ed una TLB L1 dati da 64-entry
 - Se c'è il miss ad L1 occorrono 6 cicli di CPU per verificare la entry nella L2 512-entry TLB.
 - Se miss in L2 allora la CPU o cerca le entry in memoria, con costo di centinaia di cicli, oppure può interrompere per delegare al SO il compito ...
- TLB sono elementi hardware a supporto del paging, gli SO devono essere progettati tenendo presente le caratteristiche di questi elementi che possono variare a seconda della piattaforma

Esercizio

- Sia dato un sistema di paginazione con una tabella delle pagine che risieda in memoria con un TLB con un hit ratio del 90%
- Se per un accesso in memoria occorrono 200 nanosecondi, quanto tempo occorrerà per ottenere il dato relativo a un indirizzo logico?
 - Si supponga trascurabile il tempo di ricerca della entry nella tabella delle pagine

Esercizio

- Sia dato un sistema di paginazione con una tabella delle pagine che risieda in memoria con un TLB con un hit ratio del 90%
- Se per un accesso in memoria occorrono 200 nanosecondi, quanto tempo occorrerà per ottenere il dato relativo a un indirizzo logico?
 - Si supponga trascurabile il tempo di ricerca della entry nella tabella delle pagine
 - Caso hit: tempo di accesso 200 ns
 - Caso miss: tempo di accesso 400 ns
 - Tempo accesso effettivo: $0.9 \times 200 + 0.1 \times 400 \text{ ns} = 220 \text{ ns}$

Protezione di Memoria

- La protezione della memoria è implementata associando dei bit di protezione per ogni frame per indicare i permessi (read-only, read-write)
 - Si possono aggiungere più bit per indicare execute-only, etc.
 - Mentre si cerca il frame si può controllare l'accesso
 - Una violazione dei permessi provoca un trap hardware

- Alle entry della page table sono associate anche bit **valid-invalid** :
 - “valid” indicata che la pagina nello spazio logico del processo è valida, quindi legale
 - “invalid” indica che la pagina non è nello spazio logico del processo
 - Ogni violazione provoca un trap al kernel
 - Il SO usa questi bit per permettere o vietare l'accesso alle pagine

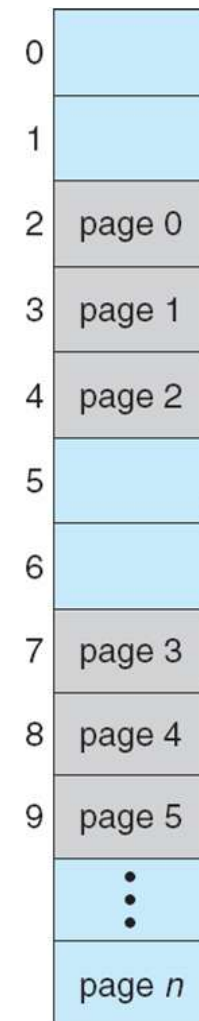
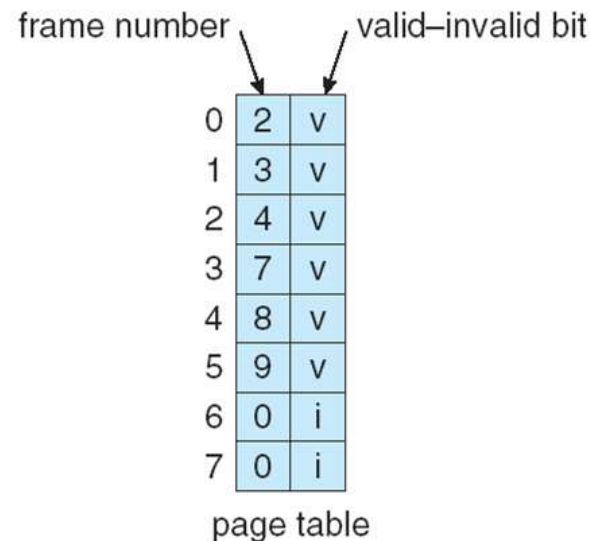
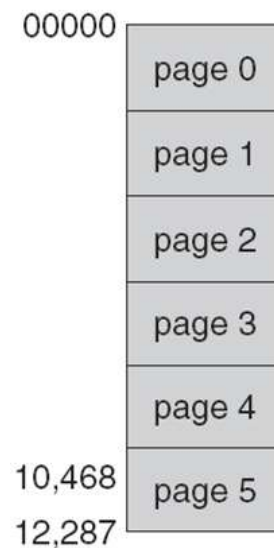
Valid (v) o Invalid (i) Bit in Page Table

Es. 14-bit address space [0,16383], processo solo in [0, 10468]

Pagine di 2KB, quindi le pagine 6, 7 non sono valide

Però pagina 5 accessibile (frammentazione interna) fino a 12287

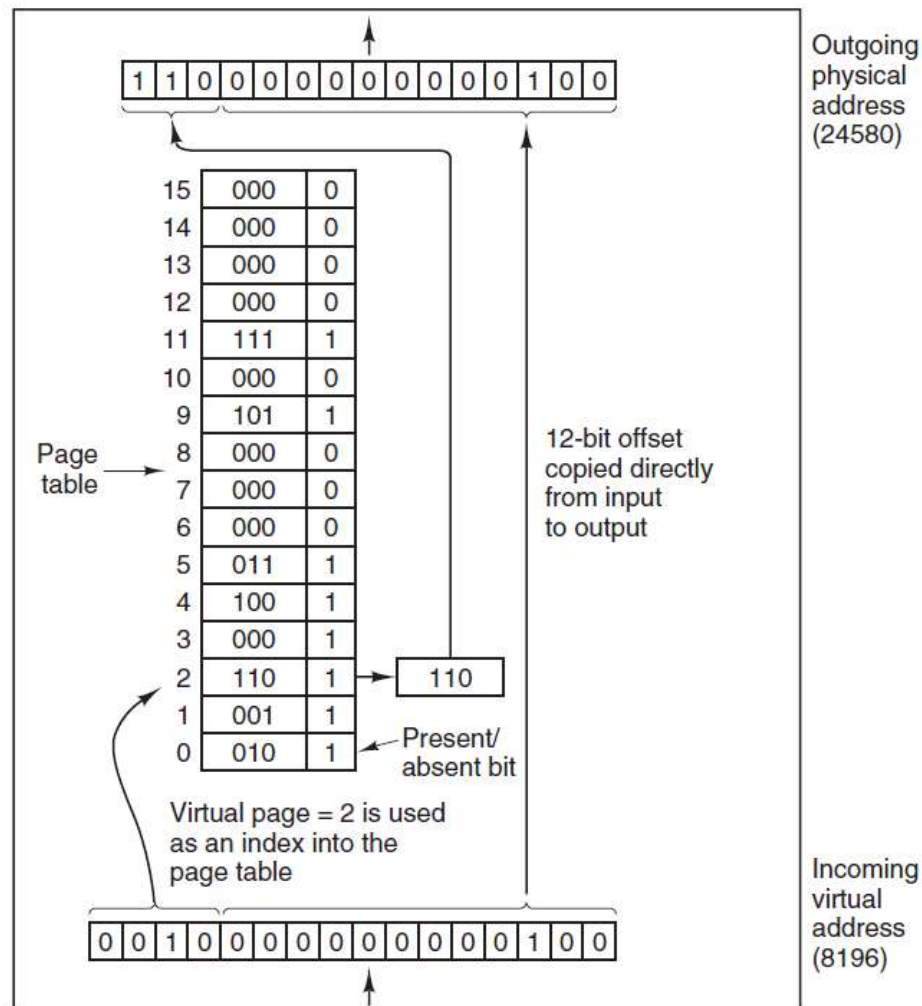
Accesso disabilitato



Si può usare il **page-table length register (PTLR)** per indicare la dimensione della tabella delle pagine e verificare se l'accesso è consentito oppure no

Present/Not Present Bit in Page Table

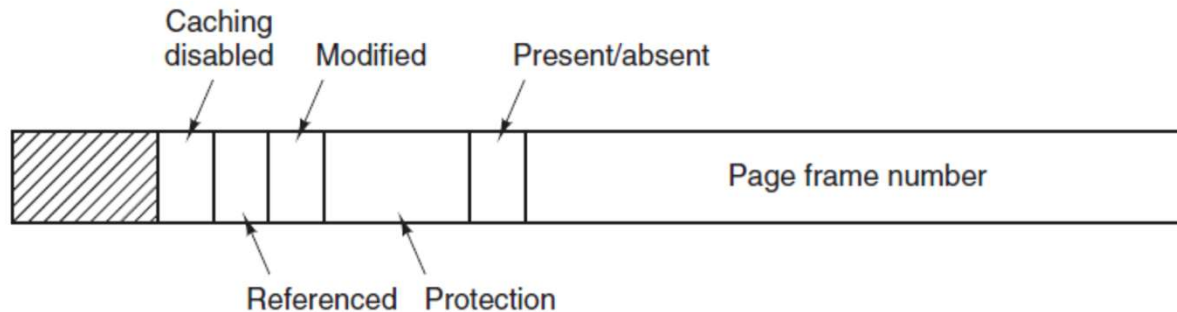
Es. 16 bit e pagine di 4 KB (2^{12}), entry tabella delle pagine?



Indirizzo virtuale 8196 tradotto nell'indirizzo fisico 24580

Entry di una Page Table

Diverse informazioni aggiuntive nella Page Table



Presente/Assente	page fault
Valido/invalid	
Protezione	read/write o only read
Modificato	dirty bit
Riferito	
Caching	(per registri dei dispositivi)

Pagine Condivise

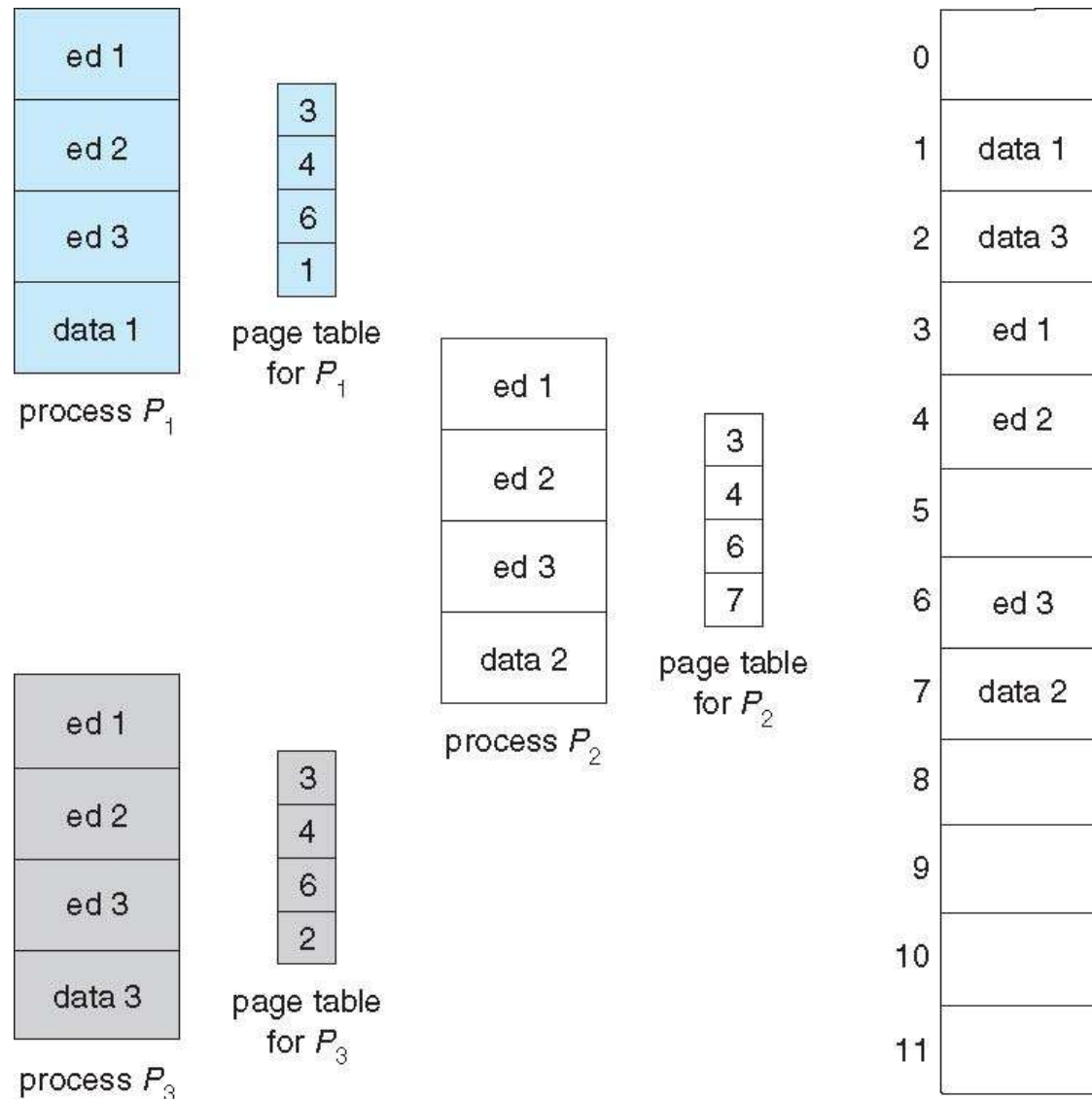
- Vantaggio del paging è la possibilità di condividere pagine fra processi
 - Particolarmente vantaggioso in architetture multiutente

- **Codice condiviso**
 - Una copia di codice read-only (**rientrante**) condivisa tra processi (i.e., text editor, compilatori, librerie) evitando duplicazioni
 - ▶ Due o più processi possono condividere lo stesso codice
 - ▶ Es. Standard C library condivisa tra processi (se 2Mb per 40 utenti, 2Mb vs 80 Mb)
 - Simile a thread multipli che condividono lo stesso spazio di processo
 - Utile anche per interprocess communication se possibile condividere pagine read-write

- **Codice e dati privati**
 - Ogni processo mantiene copia separata del codice e dei dati
 - Le pagine per codice privato e dati può apparire ovunque nello spazio degli indirizzi logici

Esempio Pagine Condivise

- Esempio codice editor condiviso da tre processi



Struttura della Tabella delle Pagine

- ❑ Moderni elaboratori supportano spazi di indirizzi logici molto estesi (da 2^{32} a 2^{64})
- ❑ Struttura per la paginazione può diventare enorme
 - ❑ Se indirizzi logici di 32-bit e dimensione delle pagine di 4 KB (2^{12})
 - ❑ La page table avrebbe più di un milione di entry ($2^{32} / 2^{12}$)
 - ❑ Se ogni entry è di 4 byte -> 4 MB di spazio di indirizzi fisici solo per la tabella delle pagine
 - ▶ Alto costo di memorizzazione
 - ▶ Non si vuole allocare in modo contiguo in memoria principale
- ❑ Metodi
 - ❑ Hierarchical Paging
 - ❑ Hashed Page Tables
 - ❑ Inverted Page Tables

Tabella delle Pagine Gerarchiche

- Suddividere lo spazio degli indirizzi logici su più tabelle delle pagine
- Una tecnica semplice è la tabella delle pagine su più livelli
 - Si pagina la tabella delle pagine
 - Il page number è a sua volta diviso in page number e offset
 - Es. con spazio degli indirizzi logici a 2^{32} e pagine di 4KB

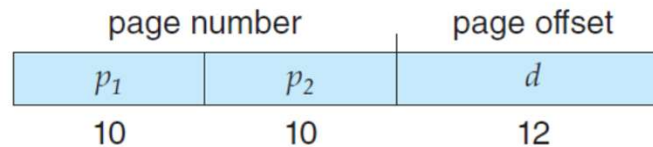
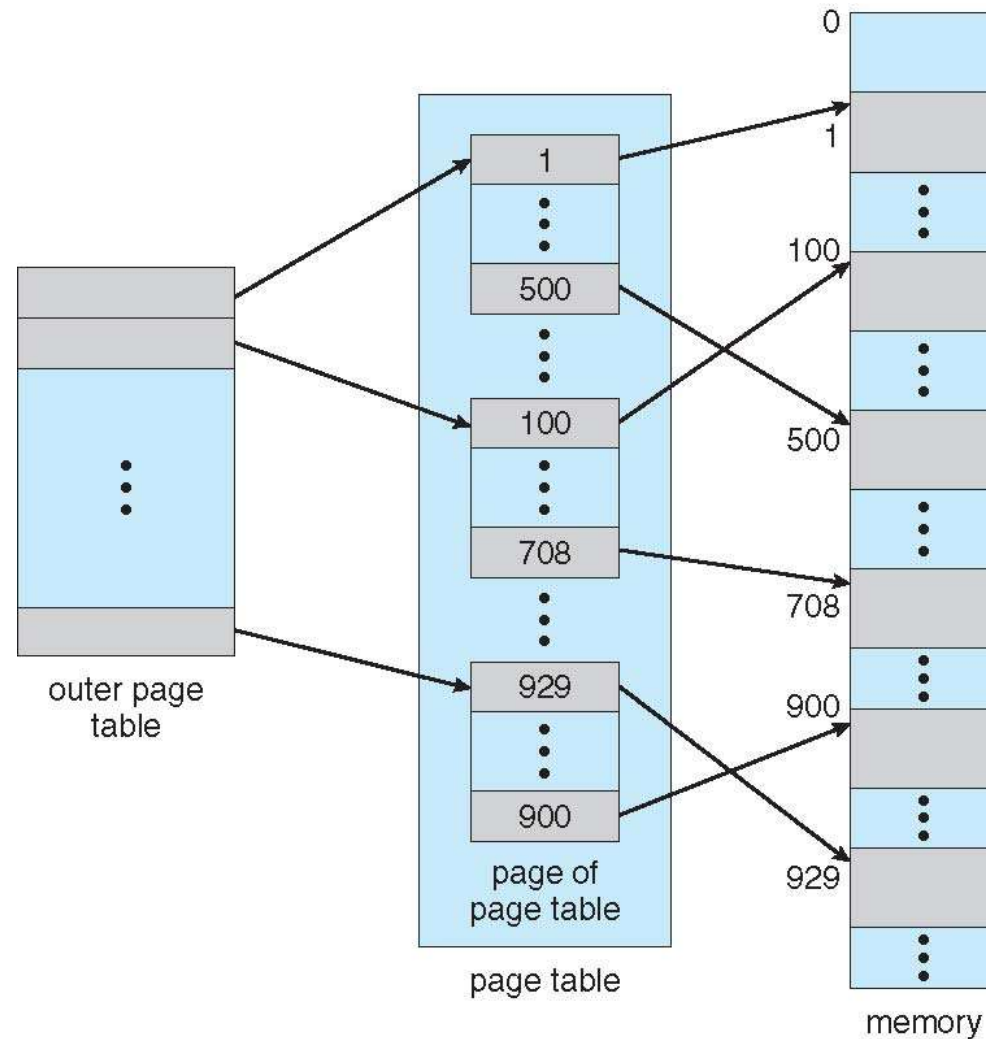
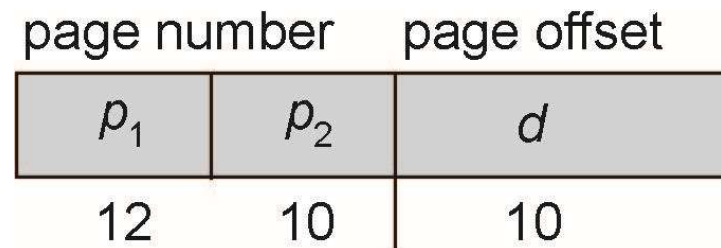


Tabella delle Pagine a Due Livelli



Esempio Paginazione a due Livelli

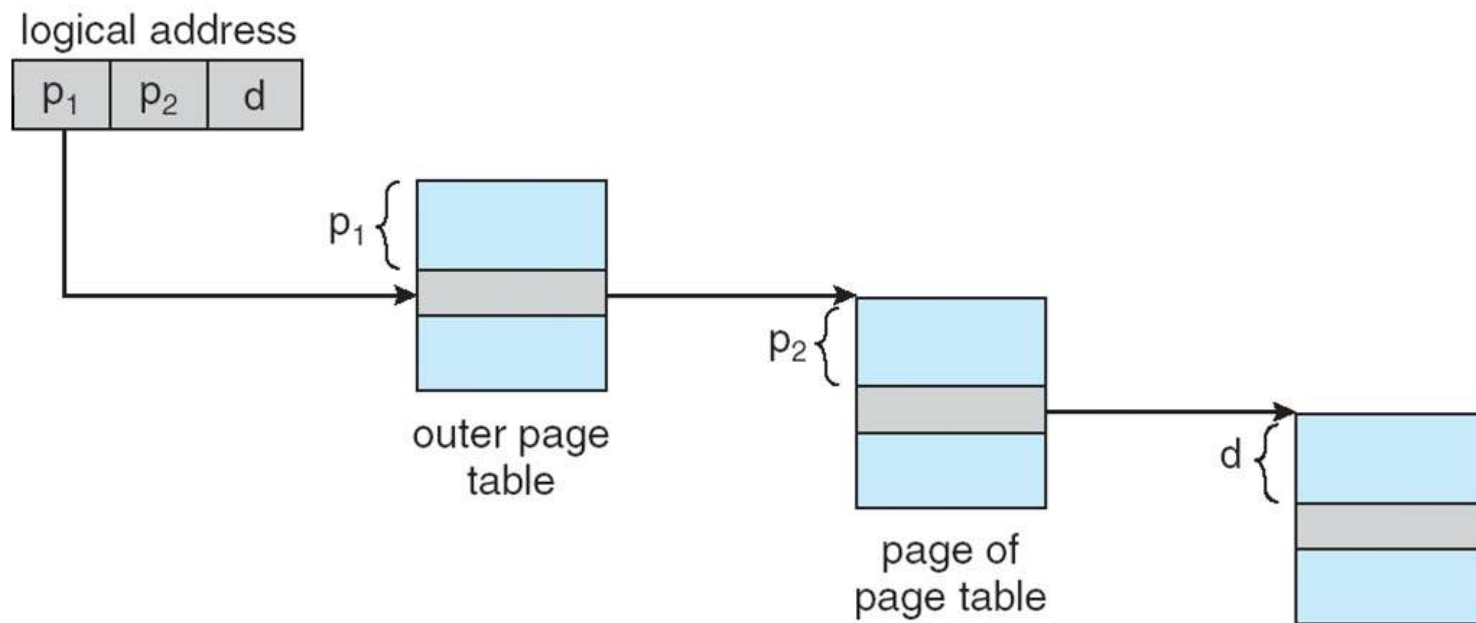
- Un indirizzo logico (su macchina a 32-bit con pagine da 1KB) diviso in:
 - page number di 22 bit
 - page offset di 10 bit
- La tabella delle pagine è paginata e il page number è ancora suddiviso in:
 - 12-bit page number
 - 10-bit page offset
- L'indirizzo logico è quindi:



- dove p_1 è indice della tabella esterna, p_2 è l'offset nella pagina della tabella delle pagine esterna
- Metodo chiamato **forward-mapped page table**

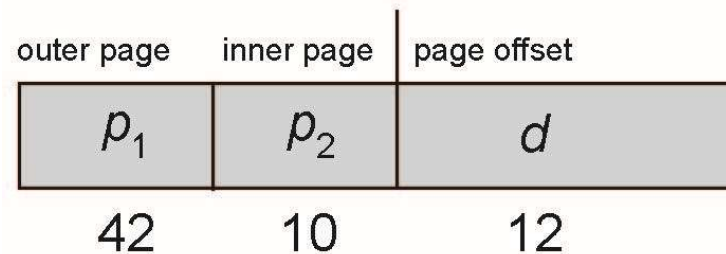
Schema di Address-Translation

- Metodo chiamato **forward-mapped page table**



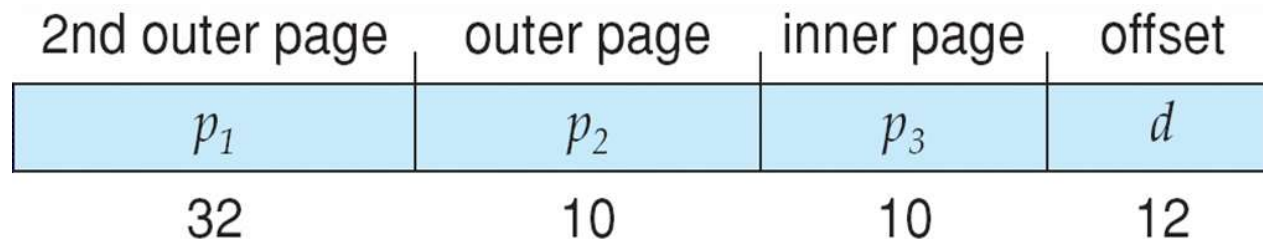
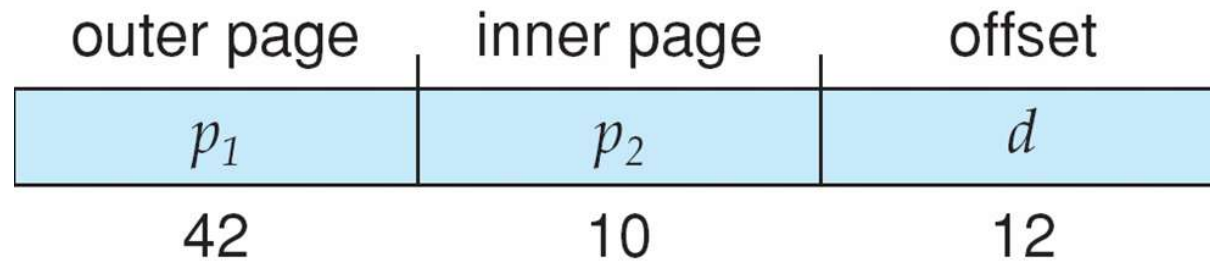
64-bit Logical Address Space

- Ma anche lo schema di paging a due livelli non è sufficiente (indirizzi 64 bit)
- Se la dimensione delle pagine fosse 4 KB (2^{12})
 - La page table avrebbe 2^{52} entry
 - Con lo schema a due livelli e pagine interne di 2^{10} con entry di 4-byte
 - L'indirizzo sarebbe



- La tabella esterna avrebbe 2^{42} entry
- Una soluzione prevede l'aggiunta di una seconda tabella esterna
- Ad esempio una seconda tabella esterna di 2^{34} byte
 - ▶ Può portare fino a 4 accessi in memoria per un singolo accesso

Schema di Paging a Tre Livelli



Sempre pagina esterna di 2^{32} entry (se 4 byte per entry allora 2^{34} byte, i.e., 16 GB)

Allora altri livelli?

Hashed Page Table

- Tipico con spazio di indirizzi > 32 bits
- Il numero di pagina virtuale viene messo in una hash page table
- Hash Table:
 - Struttura dati per implementare array associativi
 - ▶ Mappa chiave su valore
 - ▶ Utilizza una funzione di hash h che traduce una chiave in un indice di una tabella

$$h : U \rightarrow \{0, 1, \dots, m - 1\}$$

Dove U è l'universo delle chiavi, mappate da h su indici sono indici di una tabella

$$T[0 \dots m - 1]$$

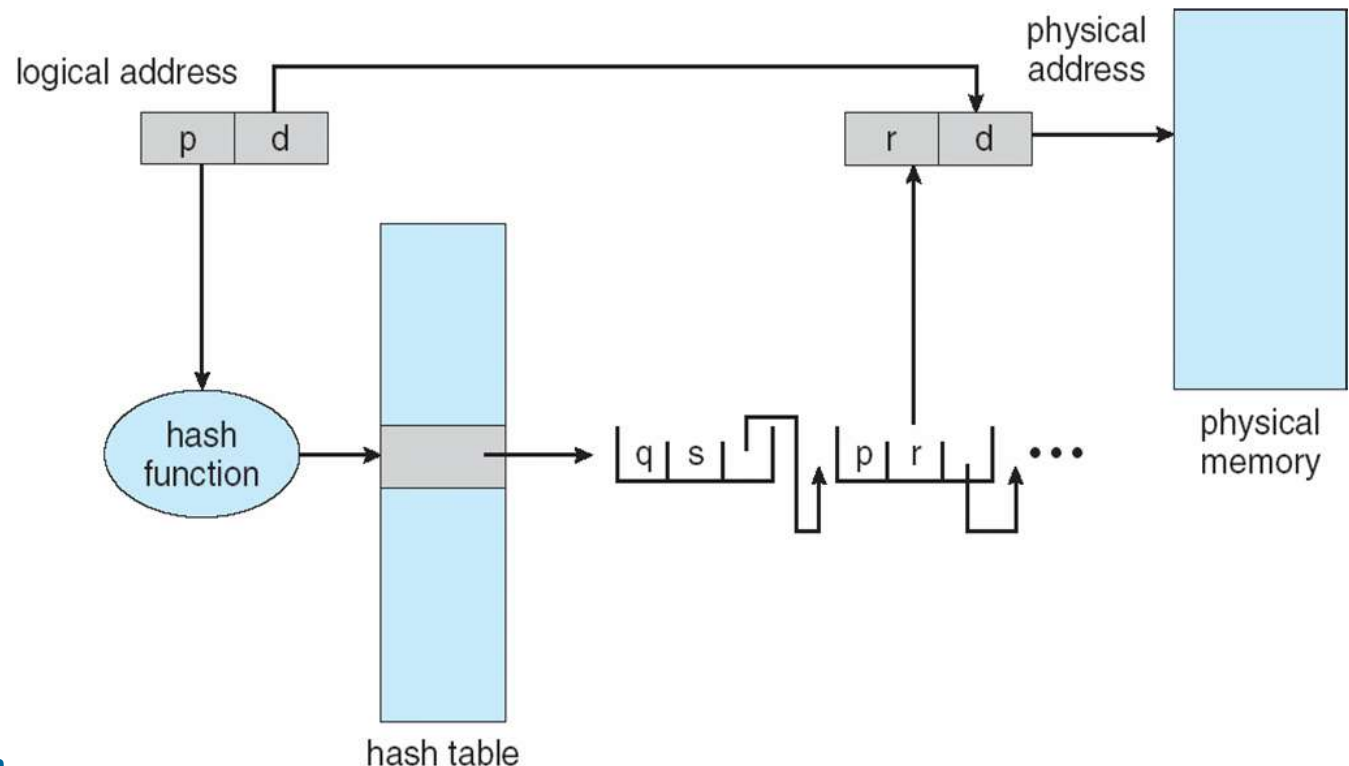
Dal momento che Hash table utile con $m \ll |U|$ sono possibili collisioni

$$h(k_1) = h(k_2)$$

Finché il numero di elementi in tabella $n < m$ sono rare
Con $n > m$ inevitabili

Hashed Page Table

- Tipico con spazio di indirizzi > 32 bits
- Il numero di pagina virtuale viene messo in una hash page table
 - La page table contiene una catena di elementi in hash sulla stessa locazione
 - Ogni elemento contiene (1) il numero di pagina virtuale (2) il valore del page frame mappato (3) un puntatore all'elemento successivo
 - I numeri di pagina virtuale sono confrontati nella catena in cerca di un match
 - Se un match è trovato viene estratto il frame fisico corrispondente



Hashed Page Table

- Tipico con spazio di indirizzi > 32 bits
- Il numero di pagina virtuale viene messo in una hash page table
 - La page table contiene una catena di elementi in hash sulla stessa locazione
 - Ogni elemento contiene (1) il numero di pagina virtuale (2) il valore del page frame mappato (3) un puntatore all'elemento successivo
 - I numeri di pagina virtuale sono confrontati nella catena in cerca di un match
 - Se un match è trovato viene estratto il frame fisico corrispondente
- Una variazione per indirizzi a 64-bit sono le **clustered page tables**
 - Simili alle hashed ma ogni entry nella hash si riferisce a molte pagine (16) invece di una sola
 - Molto utile per spazi di indirizzi **sparsi** (dove i riferimenti in memoria sono non contigui e sparpagliati)

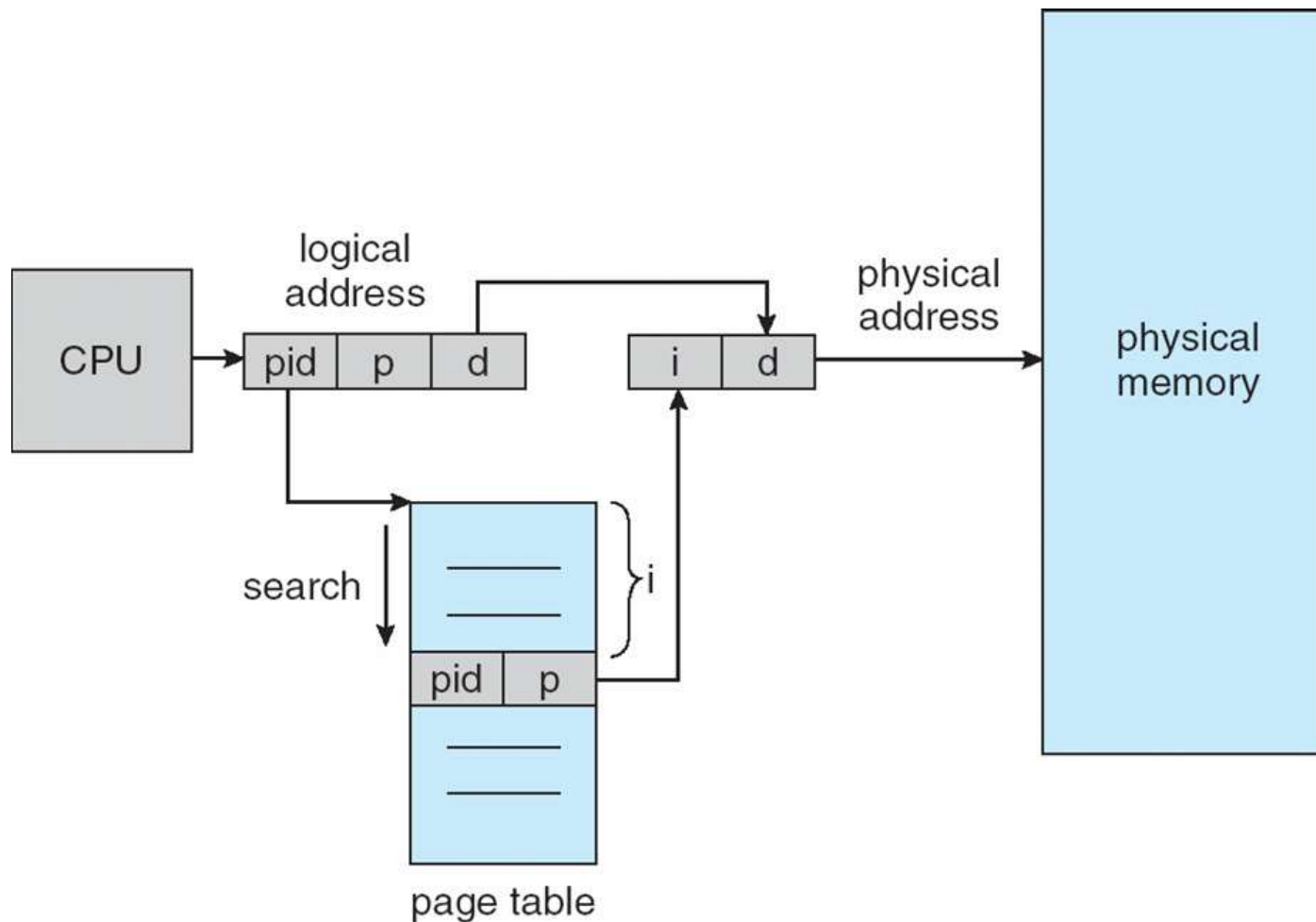
Tabella delle Pagine Invertite

- Invece di avere ogni processo con una page table e tener traccia di tutte le pagine logiche si tracciano le pagine fisiche (es. IBM RT)
 - Una entry per ogni pagina reale di memoria
 - Le entry consistono dell'indirizzo virtuale della pagina contenuta in memoria reale con informazione sul processo che possiede quella pagina

`<process-id, page-number, offset>`

- Minore memoria per le tabelle delle pagine, ma aumenta il tempo necessario per cercare la tabella quando c'è un riferimento ad una pagina
- Usata da 64-bit UltraSPARC e PowerPC

Architettura delle Tabella delle Pagine invertite



p non è più indice, ricerca del pid

Tabella delle Pagine Invertite

- Minore memoria per le tabelle delle pagine, ma aumenta il tempo necessario per cercare la tabella quando c'è un riferimento ad una pagina

`<process-id, page-number, offset>`

- Hash table per limitare la ricerca ad una - o poche - entry della tabella
 - Due accessi in memoria (uno per accedere ad hash table)
 - TLB può accelerare l'accesso (prima di accedere ad hash table)
- Più complesso implementare la memoria condivisa (shared memory)
 - Un mapping solo di un indirizzo virtuale all'indirizzo fisico condiviso
 - Un riferimento da parte di altro processo genera il fallimento ed eventuale rimpiazzo del processo che insiste sul segmento

Oracle SPARC Solaris

- Esempio di SO a 64-bit con HW strettamente integrato
 - Obiettivi: efficienza e basso overhead
- Basato su hashing, ma più complesso
- Utilizza due hash table
 - Una per il **kernel** ed una per i **processi utente**
 - Ognuna mappa indirizzi da memoria virtuale su memoria fisica
 - Ogni entry rappresenta un'area contigua di memoria virtuale mappata
 - ▶ Non una entry di hash-table separata per ogni pagina
 - ▶ Ogni entry ha un indirizzo base e uno span (che indica il numero di pagine che la entry rappresenta)

Oracle SPARC Solaris

- La ricerca di una traduzione direttamente su hash-table inefficiente
 - Si usa Translation Lookaside Buffer (TLB)
- TLB mantiene le Translation Table Entries (TTEs) per una ricerca rapida:
 - Un insieme di TTEs è anche mantenuto in un Translation Storage Buffer (TSB)
 - ▶ Include entry per le pagine con accesso recente
- Un riferimento ad un indirizzo virtuale porta alla ricerca in TLB
 - Se c'è il miss, si scandisce via hardware la TSB cercando la TTE corrispondente a quell'indirizzo (TSB walk)
 - ▶ Se si trova il match la CPU copia la TSB entry nella TLB e si completa la traduzione
 - ▶ Se non si trova il match il kernel interrompe per cercare nella hash table
 - Il kernel poi crea una TTE dalla hash table e la mette in TSB, l'interrupt handler ritorna il controllo alla MMU che completa la traduzione dell'indirizzo